MINISTRY OF EDUCATION AND RESEARCH

TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

Assignment 3

Software Design

Author: Ghile Patricia-Maria
Group: 30433

Faculty of Automation and Computer Science

April 2026

Contents

1 Problem definition **2**

2 Analysis phase - Use cases & requirements **2**
2.1 Functional requirements 2
2.2 Use cases 3

3 Design phase **3**
3.1 Architecture overview 3
3.2 Entity-relationship diagram 4
3.3 Design patterns 4
3.3.1 1. Command Pattern 4
3.3.2 2. Template Method Pattern 4
3.3.3 3. Singleton Pattern 5
3.3.4 4. Observer Pattern 5
3.4 Class diagram 5
3.5 Sequence diagram 6

4 Implementation phase - Application **7**
4.1 Tools and technologies 7
4.2 Application structure 7

5 Testing and validation **7**

1 Problem definition

The objective of this project is to evolve the **Eurovision Management System** from a monolithic desktop application into a distributed **Microservices-based architecture**. The domain remains focused on managing the Eurovision Song Contest catalog, but the system is now decomposed into independent services to improve scalability and maintainability.

This phase focuses on architectural decoupling by integrating fundamental **Design Patterns** (Singleton, Command, Template Method, and Observer) and implementing the **Command-Query Responsibility Segregation (CQRS)** principle. The system now communicates via synchronous **HTTP/REST** and features a clean separation between data modification and data retrieval.

The following sections detail the development phases:

- **Analysis phase:** Defines the functional requirements for a distributed system, including REST endpoints and CQRS separation.
- **Design phase:** Details the microservices architecture, the independent database schemas for each service, and the implementation of specific Design Patterns.
- **Implementation phase:** Describes the technology stack (Javalin, Maven) and the modular project structure.
- **Testing and validation:** Outlines the Unit Testing strategy for each service and API verification.

Deliverables: Source code (GitHub Pull Request), independent PostgreSQL schemas, and this documentation.

2 Analysis phase - Use cases & requirements

2.1 Functional requirements

The system has been refactored into two independent services: **UserService** and **CatalogService**. The following requirements are implemented:

- **CQRS Separation:** Operations are split into Commands (Create, Update, Delete) and Queries (Read, Search, Filter).
- **RESTful API:** The GUI is replaced by a set of HTTP endpoints, allowing programmatic access to the Eurovision catalog.
- **Data Export (Template Method):** Users can export catalog data into CSV or JSON formats through a structured template-driven process.
- **Role-based Access:** Authorization is handled via HTTP headers, ensuring only Admins or HODs can execute specific Command operations.

2.2 Use cases

The actors (Visitor, HOD, Admin) now interact with the system via an API client.

- **Actor:** Authenticated User (Admin/HOD).
- **Task:** Modifying data via the Command Pattern.
- **Result:** The user sends a POST/PUT/DELETE request; the system encapsulates the action into a Command object and executes it, triggering a notification upon success.

- **Actor:** Any User.
- **Task:** Querying or Exporting data.
- **Result:** The user requests data via a GET request; for exports, the system uses the Template Method to generate and save the file in the requested format.

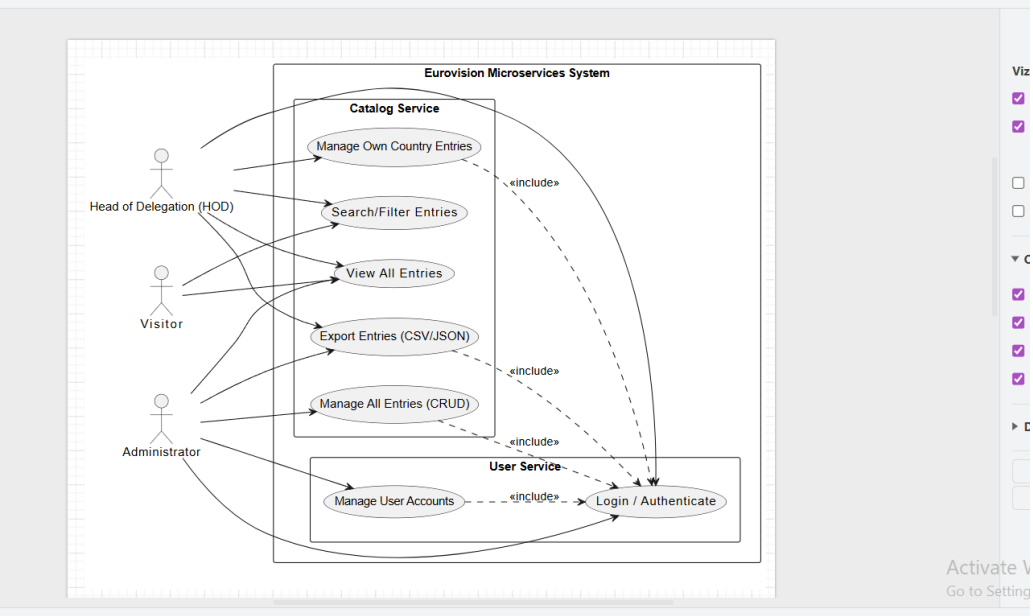

Figure 1: Microservices Use Case Diagram

3 Design phase

3.1 Architecture overview

The system follows a **Microservices Architecture**. Each service is an independent project with its own Controller, Service, and Repository layers. Communication between the user and the services is handled via REST, while the **CQRS** pattern ensures that read and write operations are logically separated.

3.2 Entity-relationship diagram

To ensure service independence, the database is split into two distinct schemas:

- `eurovision_users`: Stores authentication and role data.
- `eurovision_catalog`: Stores entries and contest metadata.

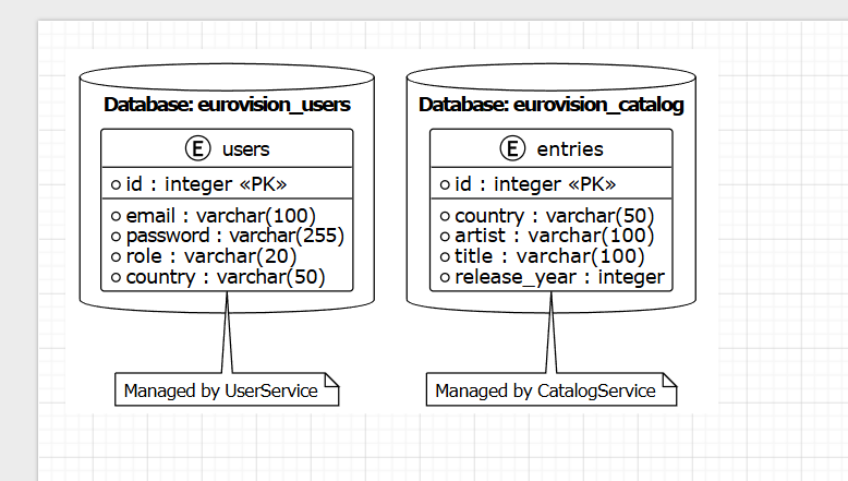

Figure 2: Decoupled Microservices ERD (One DB per Service)

3.3 Design patterns

3.3.1 1. Command Pattern

- **Category:** Behavioral
- **Problem it solves:** It decouples the HTTP Controller from the Service implementation. Every modification request (POST, PUT, DELETE) is encapsulated as a `Command` object, allowing for uniform execution and easier logging or undoing of actions.
- **Classes involved:** `Command` (Interface), `CreateEntryCommand`, `UpdateEntryCommand`, `DeleteEntryCommand`.

3.3.2 2. Template Method Pattern

- **Category:** Behavioral
- **Problem it solves:** Used for exporting data. A base class (`DataExportTemplate`) defines the fixed steps of the algorithm (fetch, transform, write), while subclasses implement only the format-specific transformation (CSV or JSON). This ensures a consistent export process.
- **Classes involved:** `DataExportTemplate` (Abstract), `CsvExporter`, `JsonExporter`, `XmlExporter`

3.3.3 3. Singleton Pattern

- **Category:** Creational
- **Classes involved:** `ConnectionFactory` and `EntryService`.
- **Purpose:** Ensures single points of access for database connections and service logic within each microservice instance.

3.3.4 4. Observer Pattern

- **Category:** Behavioral
- **Purpose:** Decouples the `EntryService` from the `NotificationService`. When a command is successfully executed, the service notifies registered observers to simulate sending email updates.

3.4 Class diagram

The class diagrams represent the internal structure of the services, highlighting the Command and Template Method integrations.

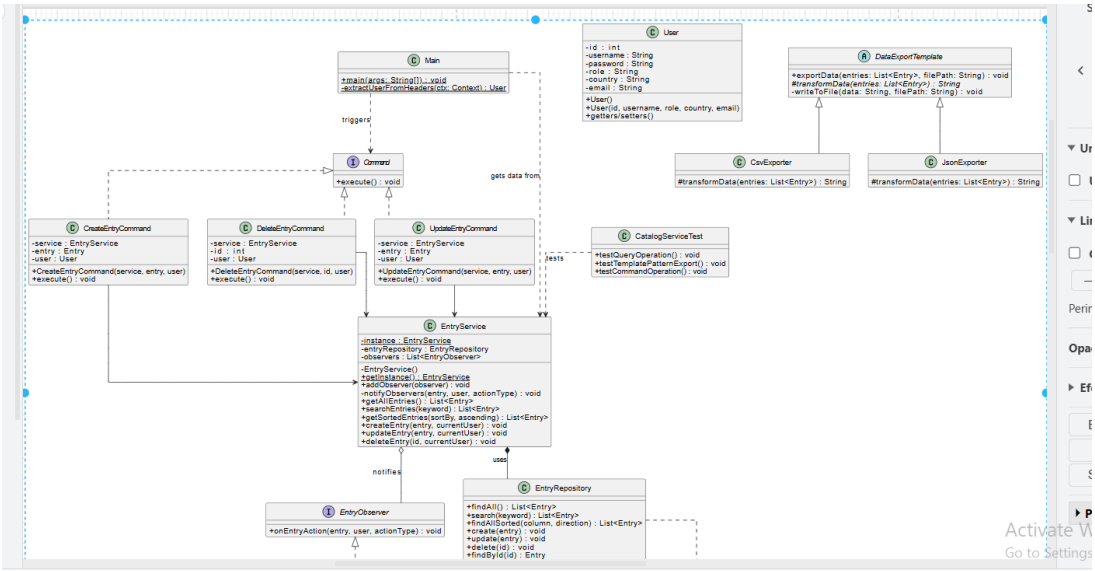

Figure 3: Catalog Service - Domain and Command Patterns

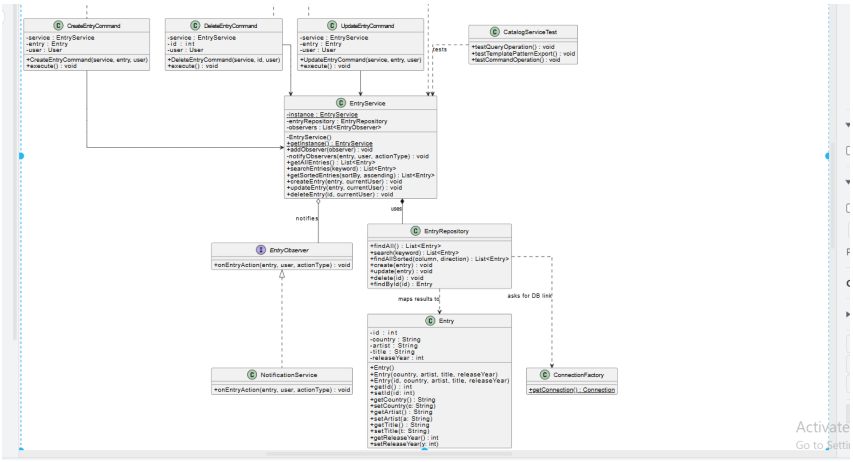

Figure 4: User Service and Shared Components

3.5 Sequence diagram

This diagram illustrates the flow of a Command execution: from the HTTP DELETE request to the Command object creation, service execution, and observer notification.

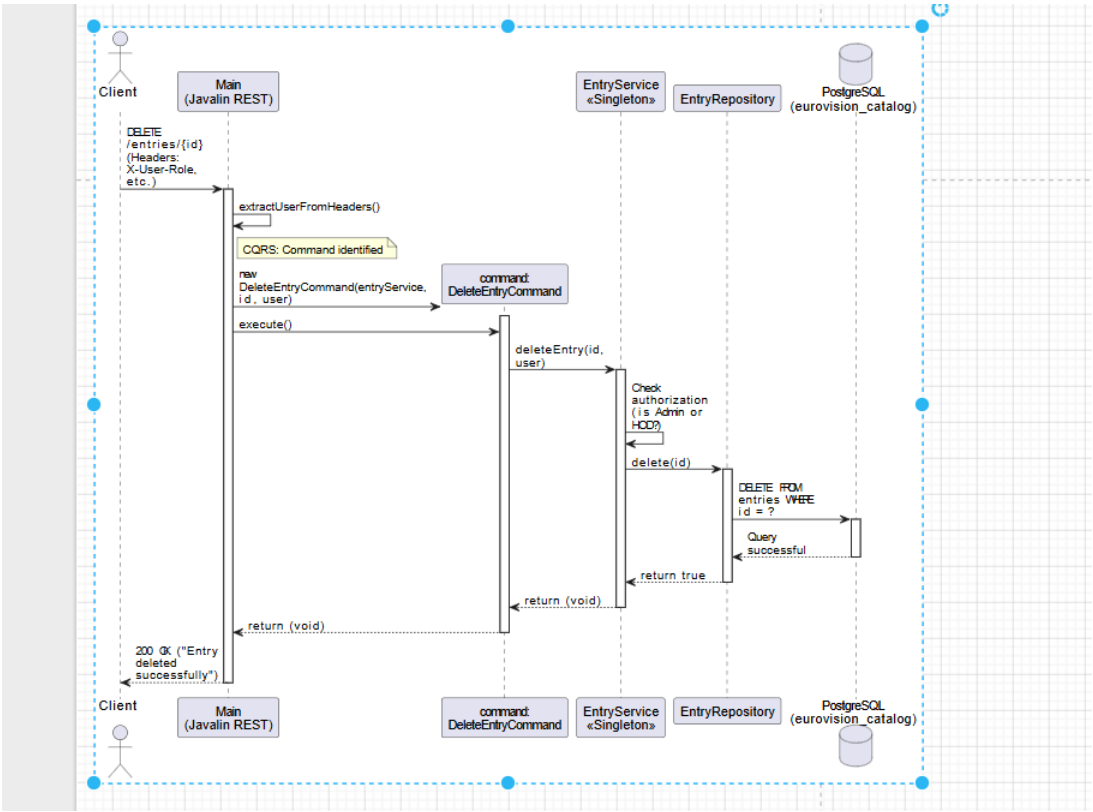

Figure 5: Sequence Diagram - Command Execution and Notification

4 Implementation phase - Application

4.1 Tools and technologies

The technology stack has been updated to support web-based microservices:

- **Java 21 & Maven**: Core language and dependency management.
- **Javalin**: Lightweight web framework for REST endpoints.
- **Jackson**: For JSON serialization and CQRS query responses.
- **PostgreSQL**: Distributed persistence (separate databases).

4.2 Application structure

Each service is a standalone module:

- `org.example.eurovision_manager.command`: Encapsulates write operations.
- `org.example.eurovision_manager.export`: Implements the Template Method for file generation.
- `org.example.eurovision_manager.model.repository`: Direct JDBC data access logic.

5 Testing and validation

Verification was conducted at two levels:

- **Unit Testing (JUnit):** Each service contains unit tests for both Command (e.g., verifying a command executes and updates the DB) and Query (e.g., verifying search logic) operations.
- **API Testing:** The HTTP/REST endpoints were validated using an HTTP client, testing for correct status codes (200 OK, 201 Created) and security constraints (403 Forbidden for unauthorized HOD actions).

References

[1] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional.

[2] Javalin Team. *Javalin - A lightweight web framework for Java and Kotlin*. `https://javalin.io/documentation` (Accessed: May 2026).

[3] PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. `https://www.postgresql.org/docs/16/index.html` (Accessed: May 2026).

[4] Martin Fowler. *CQRS - Command Query Responsibility Segregation*. `https://martinfowler.com/bliki/CQRS.html` (Accessed: May 2026).